ASSESSMENT - PSEUDO CODE WRITING.

Q1. DESIGNING CLASS HIERARCHY (PSEUDO CODE)

```
CLASS Person
    DATA
        name
        age
    METHOD Person(name, age)
        this.name ← name
        this.age ← age
    END METHOD
END CLASS

CLASS Student Inherits person
    DATA
        registerNumber
        marks[5]
    METHOD Student(name, age, registerNumber,
                                    marks[5])
        SUPER(name, age)
        this.registerNumber ← registerNumber.
        this.marks ← marks
    END METHOD
    METHOD calculateAverage()
        total ← 0
        FOR i ← 0 TO 4
            total ← total + marks[i]
        END FOR
        RETURN total/5
        END METHOD
END CLASS
```

```
CLASS ResearchStudent INHERITS Student
    DATA
        researchArea.
    METHOD ResearchStudent(name, age, registerNumber,
                                    marks[5], researchArea)
        SUPER(name, age, registerNumber, marks)
        this.researchArea ← researchArea
    END METHOD

    METHOD displayCompleteDetails()
            PRINT "Name: " + name
            PRINT "Age: " + age
            PRINT "Register Number: " + registerNumber
            PRINT "Average Marks: " + calculateAverage()
            PRINT "ResearchArea: " + researchArea
    END METHOD
END CLASS

BEGIN
        r.s ← NEW ResearchStudent("Anitha", 24, "R1023",
                                    [88, 91, 79, 85, 93], "Machine learning")
        r.s displayCompleteDetails()
END
```

Q2 : COMPLETE THE MISSING PSEUDO CODE :

```
BEGIN
    READ N
    READ ARRAY A
    FOR i ← 0 TO N-1
        FOR j ← 0 i+1 TO N-1
            IF A[i] == A[j]
                A[j] ← -1
            ENDIF
        END FOR
    END FOR
```

```
FOR i ← 0 TO N-1
    IF A[i] != -1
        PRINT A[i]
    END IF
END FOR

END
```

Q3 : DEBUG THE PSEUDO CODE

```
CLASS AREA
        METHOD area(length)
                RETURN length* length
        END METHOD

        METHOD area(length, breadth)
                RETURN length * breadth
        END METHOD
END CLASS
```

Q4: Trace the execution.

```
BEGIN
    SUM ← 0
    FOR i ← 1 TO 5
        IF i MOD 2 == 0
            Sum ← Sum + i
        ELSE
            Sum ← Sum - i
        ENDIF
    END FOR
    PRINT SUM
END
```

output:
-3

Q5:

```
BEGIN
    READ N
    READ ARRAY A
    first ← -INFINITY
    Second ← -INFINITY
    FOR i ← 0 TO N-1
        IF A[i] > first
            second ← first
            first ← A[i]
        ELSE IF A[i] > second AND A[i] != first
            Second ← A[i]
        END IF
    END FOR
    IF Second == -INFINITY
        PRINT "No second largest distinct element
                                        exist"
    ELSE
        PRINT second
    END IF
END
```

Q6: Convert problem statement to Pseudo code:

```
INTERFACE PaymentMethod.
        METHOD pay(amount)
END INTERFACE

CLASS CreditCard IMPLEMENTS PaymentMethod
        METHOD pay(amount)
                PRINT "PAID" + amount + "using Creditcard"
        END METHOD.
END CLASS
```

```
CLASS UPI IMPLEMENTS PaymentMethod
    METHOD pay(amount)
        PRINT "paid" + amount +
                "using UPI"
    END METHOD
END CLASS
CLASS NetBanking IMPLEMENTS PaymentMethod
    METHOD payCamount)
        PRINT "paid" + amount +
                "using Netbanking"
    END METHOD
END CLASS
BEGIN
    READ choice
    READ amount
    PaymentMethod method
    IF choice == "creditCard"
        method ← new creditCard()
    ELSE IF choice == "UPI"
        method ← NewUPI()
    ELSE IF choice == "Net Banking"
        method ← New NetBanking()
    END IF
    method.pay(amount)
END.
```

Q7: Runtime Polymorphism:

```
CLASS Animal
    METHOD Speak()
        PRINT "Animal"
    END METHOD
END CLASS

CLASS Dog INHERITS Animal
    METHOD Speak()
        PRINT "Barks"
    END METHOD
END CLASS

Animal obj ← New Dog()
Obj.Speak()
```

Output: Barks

Q8: Find Logical Errors (constructors)

```
CLASS Employee
    DATA
        id
        name
    CONSTRUCTOR (empId, empName)
        id ← empId
        name ← empname
    END CONSTRUCTOR
END CLASS
BEGIN
    e ← NEW Employee(101, "Ravi Kumar")
END
```

Q9 → Case study: vehicle management

```
CLASS Vehicle
    DATA
        vehicleNO
        OwnerName
    METHOD Vehicle(vehicleNo, ownerName)
        this.VehicleNO ← VehicleNO
        this.ownerName ← ownerName
    END METHOD
    METHOD display()
        PRINT "vehicle NO: " + vehicleNO
        PRINT "Owner Name: " + ownerName
    END METHOD
END CLASS

CLASS Car INHERITS Vehicle
    DATA
        fuelType
    Method car (vehicleNO, OwnerName, fuelType)
        SUPER (vehicleNO, ownerName)
        this.fuelType ← fuelType
    END METHOD
END CLASS

CLASS Truck INHERITS Vehicle
    DATA
        loadCapacity
    METHOD TRUCK (vehicleNO, OwnerName,
                  loadCapacity)
        Super(vehicleNO, ownerName)
        this.loadCapacity ← loadCapacity
    END METHOD
```

```
    METHOD display()
        SUPER.display()
        PRINT "Load Capacity: " + loadCapacity
    END METHOD
END CLASS

BEGIN
    READ N
    Vehicle Vehicles[N]
    FOR i ← 0 TO N-1
        READ type
        READ vehicleNo, ownerName
        IF type == "car"
            READ fuelType
            Vehicles[i] ← New Car(vehicleNo,
                                  ownerName, fuelType)
        ELSE IF type == "Truck"
            READ loadCapacity
            vehicles[i] ← New Truck(vehicleNo,
                                    ownerName, loadCapacity)
        ENDIF
    END FOR
    FOR i ← 0 TO N-1
        vehicles[i].display()
    ENDFOR
END
```